

QA LEADERSHIP ACADEMY

Automation Candidate Matrix

Decide what to automate, at which level of the pyramid — and, just as importantly, what to leave manual.

Purpose

This matrix replaces "automate everything" with a defensible, scored decision about each candidate test or check. It scores five criteria — risk, execution stability, business value, feedback speed and maintenance cost — and turns the total into two decisions: whether to automate at all, and if so at which level (unit, API or UI).

When to use it

- When planning an automation backlog and you need to sequence it by value.
- When a stakeholder asks for a specific journey to be automated and you need to say yes, no, or "yes but at API level".
- When triaging an inherited suite to decide what deserves to be kept, moved down the pyramid, or retired.

Instructions

1. List each candidate — a journey, endpoint or check, not "the whole app".
2. Score the five criteria 1–5 using the anchors below. Note that maintenance cost is scored inversely: a high maintenance burden scores low.
3. Total the five scores (max 25). Use the decision bands to set automate / do not automate.
4. For anything you will automate, use the level guide to place it at the lowest level that still gives real confidence.
5. Record a reason for every "do not automate" — a deliberate manual/exploratory decision is a valid, defensible outcome.

Scoring anchors

- Risk (1–5): consequence of failure. 5 = payment/data-loss/security; 1 = cosmetic.
- Execution stability (1–5): how deterministic the check is. 5 = stable inputs/outputs; 1 = timing-dependent, animation-heavy, third-party UI.
- Business value (1–5): how central to revenue or the customer promise. 5 = core journey run constantly; 1 = rare edge case.

- Feedback speed value (1-5): how much you gain from this running fast and often. 5 = wanted on every PR; 1 = fine to run occasionally.
- Maintenance cost, inverse (1-5): 5 = cheap and stable to maintain; 1 = brittle, changes every sprint.

Worked example

Applied to a sample of Northstar journeys, deciding what to automate and where:

Candidate	Risk	Stability	Value	Feedback	Maint. (inv)	Total	Decision & level
Payment authorisation & capture	5	4	5	5	3	22	Automate — API first, thin UI smoke
Login / session handling	4	4	5	5	4	22	Automate — API + one UI happy path
Discount-code calculation	4	5	4	4	5	22	Automate — unit + API
B2B partner API contract	5	5	4	4	5	23	Automate — API contract tests
Full checkout end-to-end via UI	5	2	5	3	2	17	Automate sparingly — 1-2 UI journeys only
Marketing landing-page layout	1	2	2	2	2	9	Do not automate — exploratory / visual review
One-off data migration screen	3	2	1	1	1	8	Do not automate — scripted manual check

The pattern is the point: high-risk, high-value payment logic scores well but its stability and maintenance scores push it down the pyramid to API and unit level, with only a thin UI smoke test — precisely the correction Northstar needs after a UI-heavy, 25%-flaky suite. The low-scoring cosmetic and one-off items are consciously left to exploratory testing, not forced into automation.

Level guide

- Prefer the lowest level that gives real confidence: unit < API < UI in cost and flakiness.
- Automate business logic at unit/API level; reserve UI automation for a handful of critical end-to-end journeys.
- If stability or maintenance scores are low, that is a signal to move down a level, not to add retries.

Blank reusable version

Candidate	Risk	Stability	Value	Feedback	Maint. (inv)	Total	Decision & level

Decision bands (total out of 25)

- 20-25 — Strong candidate: automate; use the level guide to place it.
- 14-19 — Conditional: automate only at the cheapest effective level, or defer.
- Below 14 — Do not automate: keep manual/exploratory and record why.

Common mistakes

COMMON MISTAKES

Treating "automate everything" as a goal; automating a high-value journey at UI level purely because that is where the failure was seen, ignoring low stability and high maintenance scores; forgetting that maintenance is scored inversely and rewarding brittle tests; and never recording a reason for "do not automate", so the deliberate decision looks like an omission.

How to interpret the results

- A high total with low stability/maintenance means "yes, but lower down the pyramid" — not "yes, at UI level".
- A cluster of high-risk, low-stability UI candidates is a signal your architecture needs test hooks or API access, not more Selenium.
- A healthy backlog has explicit "do not automate" decisions; if everything scores an automate, your anchors are too generous.
- Re-score when the product changes: today's brittle UI check may become a cheap API check once an endpoint exists.

INSIDE STLC PRO TIP

The most senior move this matrix enables is a confident "no". Being able to say "we deliberately do not automate the marketing pages — the risk is cosmetic and the maintenance cost is high, so exploratory review is the better investment" signals judgement far more than a bigger test count ever will.